

AgriSense AI: Early Detection & Precision Treatment to Safeguard Global Yields



Submitted by

Syed Ashiq Ali (F22BINFT1M01139)

Submitted to

Mr. Zaman Ali

Department of Information Technology

Faculty of Computing

The Islamia University of Bahawalpur

Summery

Here the author will write summery of the document. They can provide a brief of the background, their design considerations, application architecture and

Table of Content

1. Application Architecture

Application Architecture for **AgriSense AI: Early Detection & Precision Treatment to Safeguard Global Yields**

*. Introduction

AgriSense AI is an advanced application aimed at improving agricultural productivity by leveraging AI for early detection and precision treatment of crop diseases and pests. The goal of the architecture is to ensure scalability, responsiveness, and effective management of agricultural data across multiple sensors and external systems. This document describes the application architecture in terms of design patterns, components, APIs, and the working principles of the system.

*. Architecture Pattern: **MVVM (Model-View-ViewModel)**

For AgriSense AI, the chosen architecture pattern is **MVVM (Model-View-ViewModel)**. This pattern is ideal for applications that require clean separation of concerns, especially when dealing with real-time data and machine learning models for predictions.

Explanation of MVVM Components:

- **Model:** Represents the core data structures and logic of the application, such as crop data, disease prediction models, sensor data, etc.
- **View:** Handles the UI elements that display information to the user (e.g., disease detection dashboard, alerts, analytics graphs).
- **ViewModel:** Acts as an intermediary between the Model and View. It fetches data from the Model, processes it, and provides it in a format suitable for display in the View. It is responsible for handling data-binding.

*. Proposed Components of Architecture

The application architecture includes several key components to ensure effective processing of sensor data and AI-driven predictions.

Key Components:

- **Sensors and Data Collection Layer:**
 - Involves data input from agricultural sensors such as weather, soil, and crop health sensors.
 - Data is stored in a cloud database for analysis.
- **Data Processing & Analysis Layer:**
 - Includes AI models that process sensor data to detect early signs of crop diseases or pest infestations.
 - This layer also provides recommendations for precision treatment.
- **User Interface Layer (UI):**
 - Provides the dashboard for farmers, showing key insights such as disease alerts, precision treatment recommendations, and yield predictions.
- **External APIs and Libraries:**
 - **External APIs:** The application might integrate with external weather services (e.g., OpenWeather API) or satellite imagery APIs for additional data (e.g., Sentinel-2 data for crop health).
 - **Libraries:** Uses machine learning libraries (e.g., TensorFlow, Scikit-learn) for disease detection models, and charting libraries (e.g., D3.js or Chart.js) for data visualization.

*. Controller Functions

The controller layer in the MVVM architecture is critical for managing user requests and connecting the view to the business logic. Below is a breakdown of the functions within the **Controllers**:

Controllers Overview:

- **DiseasePredictionController:**
 - **Function 1:** Collect data from sensors (e.g., temperature, humidity, soil moisture).
 - **Function 2:** Call the AI disease detection model and pass sensor data to it for prediction.
 - **Function 3:** Fetch external weather API data to adjust predictions based on environmental factors.
 - **Function 4:** Send alerts to the user if a disease or pest infestation is detected.
- **TreatmentRecommendationController:**
 - **Function 1:** Analyze disease prediction results to provide precision treatment recommendations.

- **Function 2:** Fetch treatment guidelines based on disease severity and crop type.
- **Function 3:** Interface with external services (e.g., suppliers of fertilizers or pesticides) to get recommended treatment products.
- **UserDashboardController:**
 - **Function 1:** Display real-time updates on the user interface, such as disease alerts, treatment recommendations, and crop yield predictions.
 - **Function 2:** Provide the user with reports on historical data and trends.
 - **Function 3:** Handle user preferences (e.g., region-based disease patterns, crop type preferences).

Interaction with Views and Models:

- The **DiseasePredictionController** interacts with the **DiseasePredictionModel** to retrieve predictions and with the **WeatherDataModel** for external weather data.
- The **TreatmentRecommendationController** interacts with the **RecommendationModel** to fetch treatment suggestions based on diseases identified by the prediction model.
- The **UserDashboardController** communicates with the **UserModel** to display relevant data to the user and update the view dynamically.

*. External APIs & Libraries

External APIs and libraries play a significant role in enhancing the capabilities of AgriSense AI by providing real-time data, satellite imagery, or weather data.

External APIs:

- **OpenWeather API:** Provides weather forecasts and current conditions. This data can help improve the accuracy of disease detection models.
- **Sentinel-2 Satellite Data API:** Satellite imagery to monitor crop health and track large-scale environmental changes.
- **CropHealth Database API:** External API that holds a database of crop diseases and pest profiles, which is crucial for early detection.

External Libraries:

- **TensorFlow:** For building and running AI models that predict crop diseases or pest infestations based on sensor data and external inputs.
- **Scikit-learn:** Used for machine learning models for data preprocessing and prediction.
- **Chart.js / D3.js:** Libraries used for creating interactive charts and data visualizations on the user dashboard.

*. UML Diagrams

Class Diagram:

A class diagram can be used to illustrate the relationships between the different classes such as the models, views, and controllers. Below is a brief overview of the class diagram:

- **DiseasePredictionController** has a one-to-one relationship with **DiseasePredictionModel**, where it sends data to the model for predictions and retrieves the results.
- **TreatmentRecommendationController** interacts with the **RecommendationModel**, which stores possible treatment options based on the disease detected.
- **UserDashboardController** communicates with the **UserModel**, which contains the user's preferences, data, and historical information.

Sequence Diagram:

A sequence diagram can explain the flow of data between the **UserInterface**, **Controllers**, and **Models**:

1. **User requests disease prediction** via the **UserInterface**.
2. **DiseasePredictionController** collects sensor data and queries external weather APIs.
3. **DiseasePredictionModel** runs the prediction and returns results to the controller.
4. **Controller** sends the prediction data back to the **View**, which updates the UI with disease alerts.
5. The user then views treatment options and recommendations on the dashboard.

*. Coding Conventions

To ensure consistency, scalability, and maintainability, the following coding conventions are adopted:

- **Naming Conventions:**
 - Classes: PascalCase (e.g., DiseasePredictionController)
 - Methods/Functions: camelCase (e.g., getDiseasePrediction)
 - Variables: camelCase (e.g., sensorData)
- **Commenting & Documentation:**
 - All major functions and classes will be well-documented with docstrings and comments explaining their purpose.
 - Functions should be self-explanatory, but complex logic will be accompanied by inline comments.
- **Code Formatting:**
 - Indentation will be consistent (4 spaces per level).
 - Use of proper error handling (try/except blocks where necessary) and logging.
 - Consistent use of brackets {} in conditionals and loops.
- **Version Control:**
 - Git is used for version control. Each feature or bug fix will be developed on its own branch, and pull requests will be reviewed before merging.

This architecture ensures a scalable, flexible, and efficient solution for early disease detection and precision treatment in agriculture.

2. Data Model Schema

This section describes the **conceptual database schema** of AgriSense AI. It explains each **data model entity**, its attributes (working properties), and its **relationships** with other entities. The schema is designed to support real-time sensor ingestion, AI-based disease detection, and precision treatment recommendations.

2.1 Database Design Overview

AgriSense AI uses a **relational database** (e.g., PostgreSQL / MySQL) for structured data and may integrate a **time-series database** (e.g., InfluxDB) for high-frequency sensor readings.

Design Goals:

- Maintain normalized data for consistency
- Support real-time and historical analysis
- Enable AI model training and inference
- Ensure scalability for large farms and multiple regions

2.2 Core Data Model Entities

2.2.1 User

Represents farmers, agronomists, or administrators using the system.

Attributes:

- user_id (PK) – Unique identifier
- name – User full name
- email – Login and communication identifier
- role – Farmer / Agronomist / Admin
- location – Geographic region
- created_at – Account creation timestamp

Relationships:

- One **User** can manage multiple **Farms**
- One **User** can receive multiple **Alerts**

2.2.2 Farm

Represents an agricultural farm registered in the system.

Attributes:

- farm_id (PK)
- user_id (FK) – Owner of the farm
- farm_name
- latitude
- longitude
- total_area

Relationships:

- One **Farm** belongs to one **User**
- One **Farm** has multiple **Fields**

2.2.3 Field

Represents a specific cultivable land segment within a farm.

Attributes:

- field_id (PK)
- farm_id (FK)
- field_name
- crop_type
- soil_type
- area_size

Relationships:

- One **Field** belongs to one **Farm**
- One **Field** has multiple **Sensors**
- One **Field** has multiple **DiseaseReports**

2.2.4 Sensor

Represents IoT devices deployed in fields.

Attributes:

- sensor_id (PK)
- field_id (FK)
- sensor_type (Soil, Weather, Leaf, Humidity)
- installation_date
- status

Relationships:

- One **Sensor** belongs to one **Field**
- One **Sensor** generates multiple **SensorReadings**

2.2.5 SensorReading

Stores real-time and historical sensor data.

Attributes:

- reading_id (PK)
- sensor_id (FK)
- temperature
- humidity
- soil_moisture
- timestamp

Relationships:

- Many **SensorReadings** belong to one **Sensor**
- Used by **AI Models** for disease prediction

2.2.6 Crop

Represents crops grown in fields.

Attributes:

- crop_id (PK)
- crop_name
- growth_duration
- ideal_temperature_range
- ideal_humidity_range

Relationships:

- One **Crop** can be planted in multiple **Fields**
- One **Crop** can be affected by multiple **Diseases**

2.2.7 Disease

Represents known crop diseases and pests.

Attributes:

- disease_id (PK)
- disease_name
- affected_crop
- symptoms
- severity_level

Relationships:

- One **Disease** affects one or more **Crops**
- One **Disease** has multiple **TreatmentPlans**

2.2.8 DiseaseReport

Represents AI-generated disease detection results.

Attributes:

- report_id (PK)
- field_id (FK)
- disease_id (FK)
- confidence_score
- detection_date

Relationships:

- One **DiseaseReport** belongs to one **Field**
- One **DiseaseReport** references one **Disease**
- One **DiseaseReport** triggers multiple **Alerts**

2.2.9 TreatmentPlan

Represents precision treatment recommendations.

Attributes:

- treatment_id (PK)
- disease_id (FK)
- treatment_type (Organic / Chemical / Biological)
- dosage
- application_method

Relationships:

- One **TreatmentPlan** belongs to one **Disease**
- One **TreatmentPlan** is recommended in many **DiseaseReports**

2.2.10 Alert

Represents notifications sent to users.

Attributes:

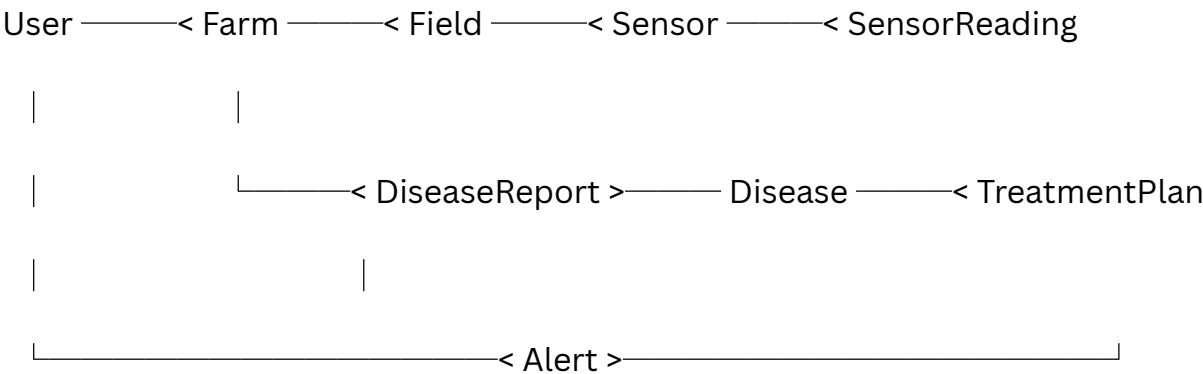
- alert_id (PK)
- user_id (FK)

- report_id (FK)
- alert_type (Disease / Weather / Treatment)
- sent_time

Relationships:

- One **Alert** belongs to one **User**
- One **Alert** is generated from one **DiseaseReport**

2.3 Entity Relationship (ER) Diagram (Conceptual)



2.4 Relationship Summary Table

Entity A	Relationship	Entity B
User	1 → Many	Farm
Farm	1 → Many	Field
Field	1 → Many	Sensor
Sensor	1 → Many	SensorReading
Field	1 → Many	DiseaseReport
Disease	1 → Many	TreatmentPlan
DiseaseReport	1 → Many	Alert
User	1 → Many	Alert

2.5 Data Model Benefits

- Supports **AI-driven early detection**
- Enables **precision treatment recommendations**
- Maintains **historical traceability** of crop health
- Scales for **large farms and global deployment**

If you want, I can also:

- Convert this into a **formal SDD-style diagram section**
- Draw a **proper ER diagram (crow's foot notation)**
- Align this schema with your **application controllers and APIs**

3. User Interface (UI) Design

1. **AgriSense AI: Early Detection & Precision Treatment to Safeguard Global Yields**
2. This section describes the User Interface (UI) design philosophy, layout strategy, screen structure, and interaction details of the AgriSense AI application. The UI is designed to be simple, intuitive, and data-driven, keeping in mind that end users may include farmers and agronomists with varying levels of technical expertise.

3. 3.1 Overall Design Philosophy

4. **3.1.1 Design Goals**
5. **Clarity:** Present complex agricultural and AI insights in a simple, understandable manner
6. **Efficiency:** Minimize steps to access critical alerts and recommendations
7. **Accessibility:** Easy readability in outdoor conditions
8. **Consistency:** Uniform layout, icons, and interactions across all screens
9. **3.1.2 Color Scheme**
10. The color scheme is inspired by agriculture and nature:

1. Color	1. Purpose
1. Green	1. Crop health, normal status
1. Yellow	1. Warning or moderate risk
1. Red	1. Disease detection or critical alert
1. Blue	1. Weather and informational elements
1. White / Light Grey	1. Background for readability

- 1.
- 2. High-contrast colors are used for alerts to ensure visibility under sunlight.
- 3. 3.1.3 Typography
- 4. Primary Font: Sans-serif (e.g., *Roboto* or *Open Sans*)
- 5. Font Size:
 - a. Headings: 18–24 px
 - b. Body text: 14–16 px
- 6. Font Weight:
 - a. Bold for alerts and critical data
 - b. Regular for descriptive text
- 7. Typography prioritizes legibility on mobile devices.
- 8. 3.1.4 Design Niche
- 9. Minimalist dashboard-style UI
- 10. Card-based layout for quick information scanning
- 11. Icon-based indicators for disease, weather, and treatment
- 12. Data visualization using charts and heat maps
- 13. 3.1.5 Screen Constraints & Responsiveness
- 14. Target Platforms:
 - a. Mobile (Android / iOS) – Primary
 - b. Tablet – Secondary
 - c. Web Dashboard – For agronomists and administrators
- 15. Responsive Design:
 - a. Uses flexible grids and scalable components

- b. Supports screen sizes from 5 inches to 15 inches
- c. Touch-friendly controls for mobile users

16. 3.2 List of Screens / Pages

17. 3.2.1 Login & Registration Screen

18. **Purpose:** Authenticate users and provide secure access.

19. **Displayed Data:**

20. **Application logo**

21. **Login / registration fields**

22. **Error or success messages**

23. **Controls Used:**

24. **Textbox: Email**

25. **Password Field: Password**

26. **Button: Login**

27. **Button: Register**

28. **Link: Forgot Password**

29. **Input Constraints:**

30. **Email must be valid format**

31. **Password minimum 8 characters**

32. **Mandatory fields validation**

33. 3.2.2 Dashboard Screen

34. **Purpose:** Provide an overview of farm health and alerts.

35. **Displayed Data:**

36. **Current crop health status**

37. **Active disease alerts**

38. **Weather summary**

39. **Field-wise health indicators**

40. **Controls Used:**

41. **Cards: Crop health summary**

42. **Icons: Alert severity**

43. **Buttons: View Details**

44. **Drop-down: Farm / Field selection**

45. **Input Constraints:**

46. **Drop-down values populated dynamically**

47. **Selection required before loading field data**

48. 3.2.3 Farm & Field Management Screen

49. **Purpose:** Manage farms and their fields.

50. **Displayed Data:**

51. **List of farms**

52. **Field details (crop type, area, soil type)**

53. **Controls Used:**

54. **Textbox: Farm name**

55. **Textbox: Field name**

- 56. **Drop-down: Crop type**
- 57. **Drop-down: Soil type**
- 58. **Button: Add / Update / Delete**
- 59. **Input Constraints:**
- 60. **Farm and field names are mandatory**
- 61. **Area must be numeric and positive**
- 62. **Crop type selection is required**
- 63. **3.2.4 Sensor Data Monitoring Screen**
- 64. **Purpose: View real-time and historical sensor data.**
- 65. **Displayed Data:**
- 66. **Temperature, humidity, soil moisture**
- 67. **Line charts for historical trends**
- 68. **Controls Used:**
- 69. **Date Picker: Start / End date**
- 70. **Drop-down: Sensor type**
- 71. **Graphs: Line and bar charts**
- 72. **Button: Refresh Data**
- 73. **Input Constraints:**
- 74. **Date range cannot exceed predefined limits (e.g., 30 days)**
- 75. **Sensor type must be selected**
- 76. **3.2.5 Disease Detection & Reports Screen**
- 77. **Purpose: Display AI-based disease detection results.**
- 78. **Displayed Data:**
- 79. **Detected disease name**
- 80. **Confidence score**
- 81. **Affected field**
- 82. **Detection timestamp**
- 83. **Controls Used:**
- 84. **Table: Disease reports**
- 85. **Button: View Report**
- 86. **Filters: Disease type, severity**
- 87. **Input Constraints:**
- 88. **Filter values must be valid predefined options**
- 89. **Confidence score displayed as percentage (read-only)**
- 90. **3.2.6 Treatment Recommendation Screen**
- 91. **Purpose: Provide precision treatment guidance.**
- 92. **Displayed Data:**
- 93. **Recommended treatment type**
- 94. **Dosage and application method**
- 95. **Safety guidelines**
- 96. **Controls Used:**
- 97. **Text Area: Treatment description (read-only)**
- 98. **Button: Acknowledge Recommendation**

99. **Button: Download PDF**

100. **Input Constraints:**

101. **No manual editing allowed**

102. **Acknowledgment required for record tracking**

103. **3.2.7 Alerts & Notifications Screen**

104. **Purpose: Show all alerts and system notifications.**

105. **Displayed Data:**

106. **Alert type**

107. **Severity**

108. **Date and time**

109. **Status (Read / Unread)**

110. **Controls Used:**

111. **List View: Alerts**

112. **Toggle: Mark as Read**

113. **Button: View Details**

114. **Input Constraints:**

115. **Alerts are system-generated (non-editable)**

116. **3.2.8 Profile & Settings Screen**

117. **Purpose: Manage user preferences and settings.**

118. **Displayed Data:**

119. **User profile information**

120. **Notification preferences**

121. **Language selection**

122. **Controls Used:**

123. **Textbox: Name**

124. **Drop-down: Language**

125. **Toggle Switch: Notifications**

126. **Button: Save Changes**

127. **Input Constraints:**

128. **Name cannot be empty**

129. **Language must be selected from supported list**

130. **3.3 UI Input Validation & Constraints Summary**

1. Control Type	1. Constraints
1. Textbox	1. Mandatory, length limits
1. Password Field	1. Minimum length, masked input
1. Drop-down	1. Predefined values only
1. Numeric Fields	1. Positive numbers only
1. Date Picker	1. Valid date range
1. Buttons	1. Disabled until inputs are valid

- 1.
- 2. **3.4 Usability & Accessibility Considerations**
- 3. Large buttons for easy touch interaction
- 4. Icons with text labels
- 5. Color-blind friendly alert indicators
- 6. Offline data caching with sync when internet is available
- 7. This UI design ensures ease of use, fast decision-making, and clear communication of AI-driven insights for safeguarding agricultural yields.
- 8. If you want, I can also:
- 9. Map each screen to controllers and APIs
- 10. Provide UI wireframe descriptions
- 11. Align this UI with mobile-first or web-only SDD format